

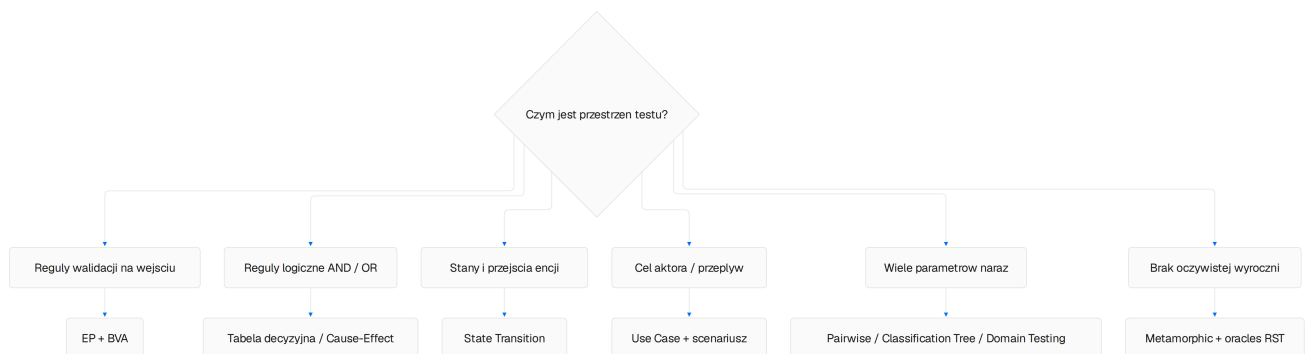
Metody testowania w praktyce — przewodnik SDET mentora

Repozytorium: Payment Quality Engineering Lab · Spring Boot 4 / Modulith · PostgreSQL 18 · Nuxt 4 + Nitro BFF · Keycloak 26. Źródła:

docs/testing/playwright-method-playbook/07-advanced-techniques-beyond-metamorphic.md; kod metod: tests-pom/methods/ (ep-bva, decision-table, state, metamorphic, pairwise, combinations); testy API: rest/*RestAssuredTest.java. Diagramy: mermaid renderowane do PNG.

Jak korzystać z przewodnika

Każda metoda ma pięć elementów: prostą definicję (max 5–7 zdań), kiedy stosować, kiedy NIE stosować, procedurę krok po kroku i przykład z naszego systemu płatności — często z autentycznym fragmentem testu (REST Assured lub Playwright). Metody uzupełniają się: techniki systematyczne projektują przypadki, metamorfizm i heurystyki RST dostarczają wyrokni tam, gdzie oczekiwanego wyniku nie da się wyliczyć z góry. Diagram poniżej mówi, od czego zacząć analizę przestrzeni.



Rysunek 1. Drzewo doboru metody: najpierw charakter przestrzeni testowej, potem technika.

1. EP + BVA — partycje równoważności i wartości brzegowe

Definicja:

Przestrzeń wejściową dzielimy na klasy równoważności zakładając, że system potraktuje wartości jednej klasy tak samo — wystarczy jedna reprezentatywka na klasę. BVA dosztukowuje analizę o punkty styku (b-1, b, b+1), bo to na brzegach implementacje zawodzą najczęściej (porównania < vs <=). Dla kwot brzegi są twarde: min/max amountMinor, capture vs authorized, threshold polityki refund. Testujemy środek każdej klasy plus sąsiadów każdego brzegu, zawsze z jawnym oczekiwanym statusem HTTP.

Kiedy stosować	Kiedy NIE stosować
wejście liczbowe/zakresowe z walidacją (amountMinor, thresholdy, TTL)	wynik zależy od kombinacji parametrów pairwise / drzewo klasyfikacji
jasne granice walidacji: min, max, zero, puste stringi	stan encji determinuje zachowanie mocniej niż wartość State Transition
chcesz tanich testów REST zamiast wolnych E2E	dane czysto prezentacyjne bez logiki walidacji

Procedura:

- 1 Wypisz parametry i typy; zaznacz te z walidacją zakresu.
- 2 Partycje: poprawna / poniżej minimum / powyżej maksimum / typy specjalne (0, puste, nie-numeryczne).
- 3 Reprezentant z partycji + sąsiedzi brzegów b-1, b, b+1.
- 4 Zapisz status HTTP i kod problem+json dla każdej próbki.
- 5 Automatyzuj w REST Assured; UI zostaje jeden przypadek dymny.

Przykład z repo — BVA kwoty capture:

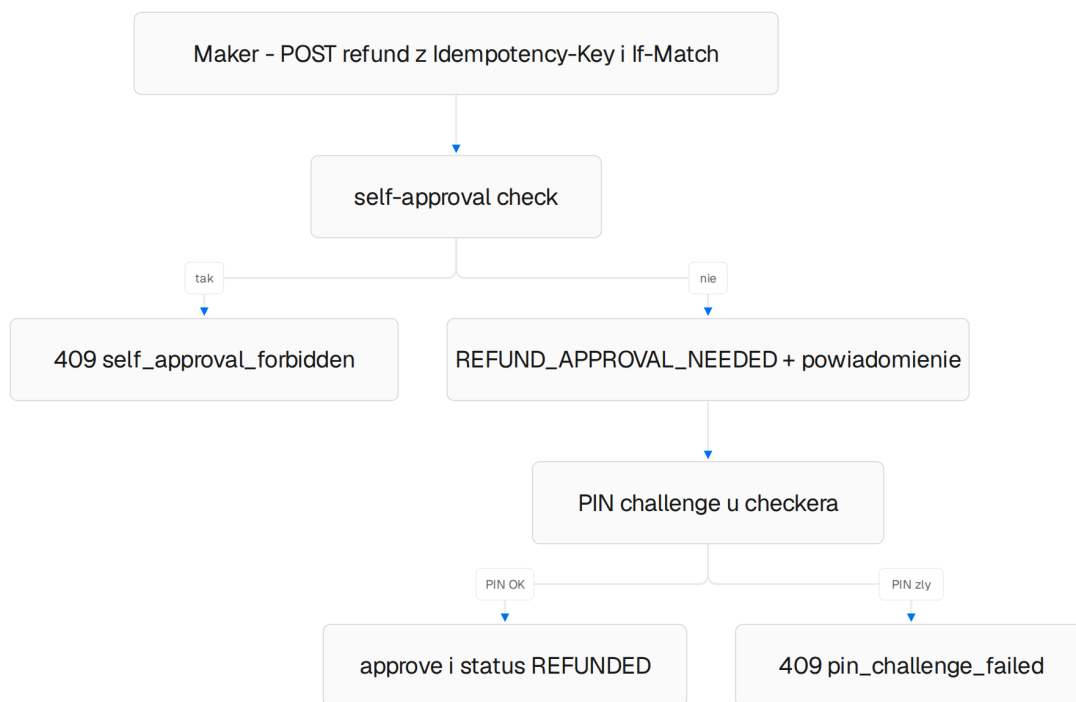
```
// tests-pom/methods/ep-bva/CaptureAmountPartitions.ts
export const captureAmountPartitions = {
  id: 'SCN-CAP-OVER',
  authorizedMinor: 2100,
  overAmountMinor: 2101, // brzeg gorny = max + 1 minor unit
  expectStatus: 422,
  error: 'capture_amount_exceeds_authorized',
} as const

// REST Assured – brzeg kwoty capture; status zamówienia bez zmian
given()
  .header("Authorization", bearer(merchantCreatorToken(merchantId)))
  .body(Map.of("amountMinor", 2101))
.when()
  .post("/api/merchants/{merchantId}/payment-orders/{id}/capture", merchantId, orderId)
.then()
  .statusCode(422)
  .contentType(ContentType.JSON)
  .body("error", equalTo("capture_amount_exceeds_authorized"))
  .body("status", equalTo("AUTHORIZED"));
```

2. Tabela decyzyjna + Cause-Effect graphing

Definicja:

Tabela decyzyjna zestawia warunki z akcjami-wynikami i pozwala skrócić kolumny, które niczego nie zmieniają. Cause-effect to forma pośrednia: przyczyny (brak If-Match, stary ETag, nielegalny stan) łączymy AND/OR/NOT i redukujemy do minimalnej tabeli. W płatnościach daje mapę odpowiedzi: 400 malformed, 428 missing, 412 stale, 409 konflikt biznesowy. Testujemy regułę, a nie endpoint — jedna tabela dokumentuje całą rodzinę zachowań.



Rys. 2. Cause-effect dual-control refundu: samozatwierdzenie i PIN jako dwa niezależne warunki.

Stosuj, gdy...	NIE stosuj, gdy...
kilka warunków logicznych determinuje jeden wynik (nagłówki × akcja × stan)	warunki to niezależne funkcje z własnymi TC
budujesz pełną mapę 400/412/428/409 dla kontraktu	iloczyn kartezjański rośnie wykładniczo bez wpływu na wynik — skracaj kolumny

Przykład z repo — macierz action × If-Match:

```

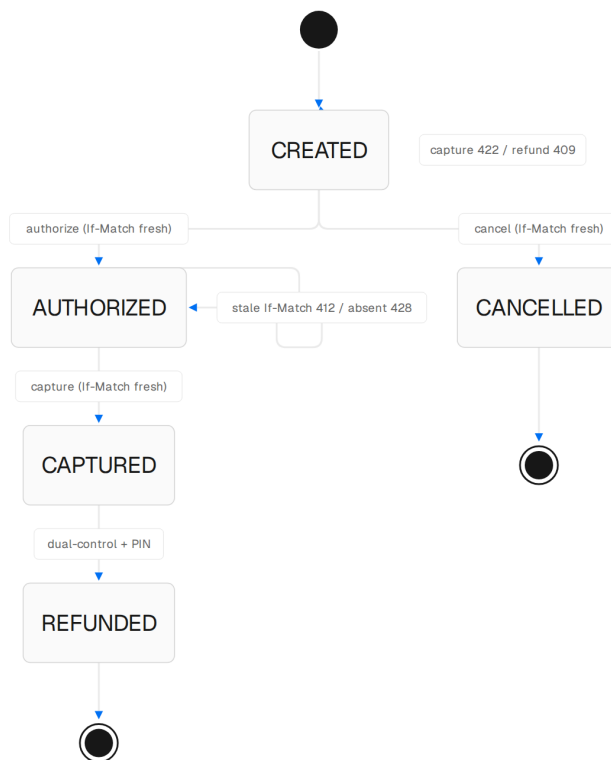
// tests-pom/methods/decision-table/IfMatchActionMatrix.ts
export const ifMatchActionMatrix = [
  { id: 'SCN-IFM-01', from: 'CREATED', action: 'cancel', ifMatch: 'absent', expectStatus: 428 },
  { id: 'SCN-IFM-02', from: 'AUTHORIZED', action: 'capture', ifMatch: 'v99', expectStatus: 412 },
  { id: 'SCN-IFM-03', from: 'CREATED', action: 'cancel', ifMatch: 'malformed', expectStatus: 400 },
  { id: 'SCN-IFM-04', from: 'AUTHORIZED', action: 'capture', ifMatch: 'absent', expectStatus: 428 },
] as const

// Playwright REST przez BFF – wykonanie wiersza tabeli end-to-end
const row = merchantIfMatchMatrix.find(r => r.testId === 'RA-M360-052')!;
const client = requireApi(api);
const created = await client.createMerchant(reference, `Race ${reference}`);
expect(created.status).toBe(201);
const stale = await client.activateMerchant(created.body.merchantId!, 'v99');
expect(stale.status).toBe(row.expectStatus); // 412
expectProblem(stale.body, row.expectStatus, 'merchant_version_mismatch');
  
```

3. State Transition Testing

Definicja:

Encję modelujemy jako automat: legalne przejścia, strażniki (If-Match, role, PIN) i stany pułapki. Test pokrywa każdą legalną krawędź co najmniej raz plus reprezentatywne krawędzie nielegalne, asertując bezzmienność stanu i właściwy kod (422/409). Pokrycie mierzymy przełącznikami: 0-switch (stany), 1-switch (krawędzie), N-switch (sekwencje). Legalne ścieżki payment_orders: CREATEDAUTHORIZEDCAPTURED, CREATEDCANCELLED, CAPTUREDREFUNDED tylko przez dual-control.



Rys. 3. Maszyna stanów payment_orders z krawędziami błędu (412/426/422/409).

Stosuj, gdy...	NIE stosuj, gdy...
encja ma jawny cykl życia (payment order, support case, merchant)	przepływ liniowy bez rozgałęzień (prosty CRUD bez stanów)
strażniki zmieniają wynik tej samej operacji (ETag, PIN, rola)	testujemy czystą walidację wejścia, nie orkestrację

- 1 Procedura: narysuj automat wypisz krawędzie legalne i nielegalne nadaj ID (SCN-LIF-01...) REST asertuje kod i bezzmienność stanu E2E pokrywa tylko krawędzie P0 z UI.

Przykład z repo — tabela krawędzi (Playwright/TS):

```
// tests-pom/methods/state/PaymentStatusMachine.ts (fragment)
export type PaymentEdge = {
  id: string; from: PaymentStatus;
  action: 'authorize' | 'capture' | 'cancel' | 'refund';
  ifMatch: 'fresh' | 'v99' | 'absent' | 'malformed';
  expectStatus: number; to: PaymentStatus;
}

export const paymentStatusEdges: readonly PaymentEdge[] = [
  { id: 'SCN-LIF-01', from: 'CREATED', action: 'authorize', ifMatch: 'fresh',
    expectStatus: 200, to: 'AUTHORIZED' },
  { id: 'SCN-LIF-03', from: 'CREATED', action: 'authorize', ifMatch: 'v99',
    expectStatus: 412, to: 'CREATED' },
  { id: 'SCN-ILL-01', from: 'CREATED', action: 'capture', ifMatch: 'fresh',
    expectStatus: 422, to: 'CREATED' },
]
```

]

4. Use Case Testing i scenariusze (atomic vs molecular)

Definicja:

Use case opisuje cel aktora: ścieżkę sukcesu plus alternatywy błędów; testujemy każdą gałąź osobno. Scenariusz Kanera to wiarygodny dzień operatora — długa sekwencja kroków biznesowych zamiast atomowych operacji. Rozróżnienie warstw jest u nas twarde: REST Assured testuje atomy (jedna operacja HTTP i jej kontrakt), a Playwright E2E molekuły (cały UC przez UI). Scenariusz uzupełnia techniki systematyczne — nie zastępuje ich.

Stosuj, gdy...	NIE stosuj, gdy...
przepływ ma aktora z celem i alternatywami (create verify act)	testujesz pojedynczą regułę walidacji — atom wystarczy
budujesz P0 happy-path dla smoke suite	scenariusz miałby setki kroków — podziel na molekuły

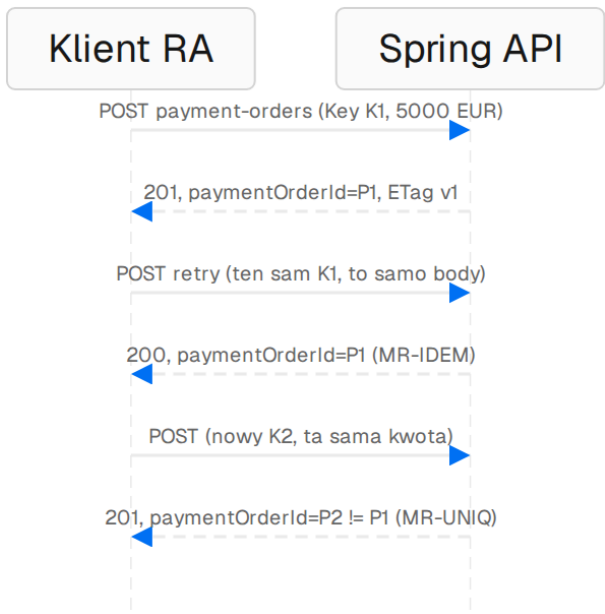
- 1 Procedura: wypisz kroki UC i gałęzie alternatywne przypisz kroki do warstw (RA=atomy, UI=molekuły) dane przez fabryki z unikalnymi referencjami testInfo asercje na widocznych efektach, nie na network internals.

```
// Playwright E2E (tests-pom/specs/merchants-concurrency.spec.ts) — molekuła „wyścig ETag”
const contextA = await browser.newContext({ storageState: pomAuthFiles.platformAdmin });
const apiB = await BffClient.create(playwright, pomAuthFiles.platformAdmin);
const appA = new App(await contextA.newPage());
await appA.merchantDetail.gotoMerchant(merchantId);           // A otwiera detal
const current = await apiB.getMerchant(merchantId);
await apiB.activateMerchant(merchantId, etagOf(current.headers!)); // B aktywuje pierwszy
const activate = await BffResponse(appA.page, { ... }); // A próbuje ze starym If-Match
// oczekiwane: A widzi Reload/412 flow, stan spójny w obu kontekstach
```

5. Metamorphic Testing (relacje MR)

Definicja:

Zamiast wyliczać oczekiwany wynik, sprawdzamy relację: transformacja wejścia pociąga przewidywalną transformację wyjścia (MR-IDEM: replay tego samego Idempotency-Key ten sam paymentOrderId i 200). Metamorfizm błyszczy tam, gdzie oracle jest drogi lub nieznany — listy, filtry, agregaty, idempotencja. Relacje komponują się: EP daje punkt odcięcia filtra, MR-FILTER sprawdza inkluzję zbiorów. W labie MR żyją w tests-pom/metamorphic jako dane deklaratywne wykonywane przez RA.



Rys. 4. MR-IDEM i MR-UNIQ: replay zwraca ten sam identyfikator; nowy klucz tworzy nowe zamówienie.

Stosuj, gdy...	NIE stosuj, gdy...
oracle trudny do wyliczenia (listy, filtry, summary, idempotent replay)	istnieje tani, precyzyjny oracle (status HTTP, kod błędu)
chcesz wykryć błąd bez znajomości dokładnych wartości	relacja jest trywialna i nic nie wykryje (x x)

```
// tests-pom/methods/metamorphic/*.ts – pięć relacji z laba
mrEtag    // MR-ETAG: GET, GET => ETag bez zmian; authorize => ETag inny
mrFilter  // MR-FILTER: zawężony minAmount ⊆ poszerzony minAmount
mrIdem    // MR-IDEM: same key + same body => ten sam paymentOrderId (200)
mrUniq    // MR-UNIQ: new key => drugi 201 i inny paymentOrderId
mrIso     // MR-ISO: merchanty tenant.admin ⊆ platform.admin; beta poza zbiorem

// REST Assured – autentyczny replay idempotentny (PaymentOrderBusinessFlowRestAssuredTest)
String paymentOrderId = paymentCreatorRequest(creatorToken, idempotencyKey, "corr-first")
    .body(body)
    .when().post("/api/merchants/{merchantId}/payment-orders", merchantId)
    .then().statusCode(201)
    .extract().path("paymentOrderId");

paymentCreatorRequest(creatorToken, idempotencyKey, "corr-second") // ten sam K1!
    .body(body)
    .when().post("/api/merchants/{merchantId}/payment-orders", merchantId)
    .then()
    .statusCode(200) // MR-IDEM: 200, nie 201
    .header("ETag", startsWith("\"v\""))
    .body("paymentOrderId", equalTo(paymentOrderId)); // ten sam zasób
```

6. Domain Testing i Classification Tree Method

Definicja:

Domain testing to EP+BVA na wielowymiarowej domenie: nie amount 0 i 101, lecz amount × waluta × status × If-Match naraz. Classification Tree Method porządkuje parametry w drzewo (rola, If-Match, from-status, klasa kwoty), a liście drzewa stają się przypadkami — widać JAK obcięto kombinacje. Na rozmowie OOM brzmi lepiej niż „zrobiłem pairwise”. W labie: refundPolicy × amountMinor × PIN required.

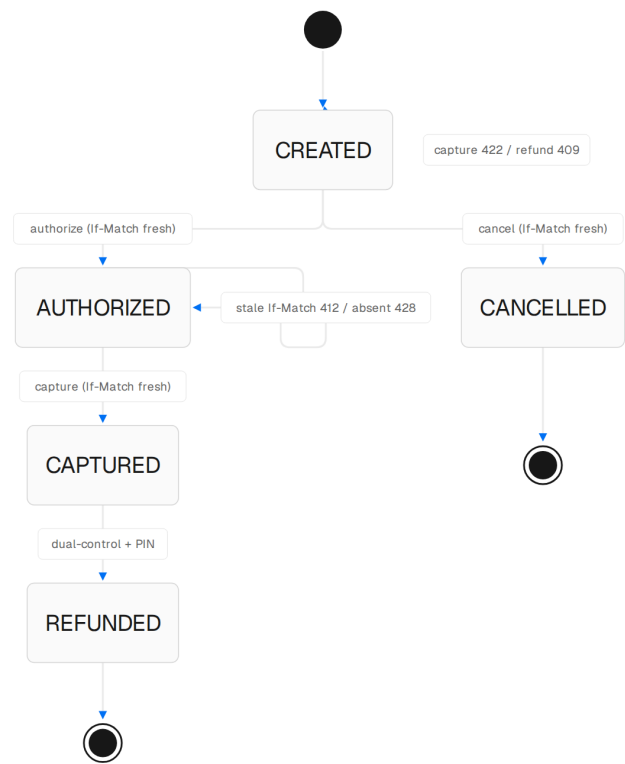
Stosuj, gdy...	NIE stosuj, gdy...
3+ wymiary wejściowe interakcyjnie wpływają na wynik	wymiary są ortogonalne i tanie w pełnym pokryciu
musisz uzasadnić cięcie kombinacji (reviewer pyta „dlaczego te TC?”)	drzewo rośnie szybciej niż budżet — tnij poziomy abstrakcji

```
// Drzewo dla dual-control refund (CTM):
// refundPolicy(MANUAL|AUTO) x amountMinor(<50|=50|>50) x PIN(required|not_required)
// Liscie = TC; granice 50 z BVA trafiają do lisci srodkowego galezi.
const tree = {
  policy: ['MANUAL', 'AUTO'],
  amountMinor: ['below_50', 'exactly_50', 'above_50'],
  pinRequired: ['required', 'not_required'],
}; // 2x3x2 = 12 lisci -> tnij: AUTO ignoruje PIN => 8 realnych TC
```

7. Rapid Software Testing: SFDPOT, HICCUPPS, tours, SBTM

Definicja:

RST to dyscyplina doboru misji i wyrokni pod niepewność, nie kolejna tabela ISTQB. SFDPOT modeluje produkt: Structure, Function, Data, Platform, Operations, Time — u nas Data=JSONB policy, Time=PIN TTL, Platform=BFF vs Spring. HICCUPPS odpowiada „skąd wiem, że to bug”: History, Image, Comparable, Claims, User Experience, Product, Purpose, Standards — np. 412 vs 409 wynika z Claims+Standards (nasza tabela HTTP), nie z zgadywania. Tours (Whittaker) i charters z time-boxem (SBTM) organizują eksplorację.



Rys. 5. Ten sam diagram stanów działa jak mapa tourów: money tour = refund+PIN; anticonformity = malformed WS.

Heurystyka	Zastosowanie w labie
SFDPOT — Product Elements	Data: JSONB polityki refund; Operations: dual-control (2 writery); Time: PIN TTL, page.clock
HICCUPPS — oracles	Claims: OpenAPI/dokument API; Standards: RFC 9110 (412 vs 428); Comparable: checkout inbox analogia
Tours — chartery	money tour: refund+PIN end-to-end; anticonformity: inject malformed WS frame do ops feed
SBTM — sesja 90 min	Charter: „Atakuj If-Match na PATCH settings; szukam 409 tam, gdzie ma być 412”

Kiedy NIE stosować:

- 1 Eksploracja nie zastępuje automatyzacji kontraktów — po sesji charter zamienia się w regresyjne *KafkaIT/*Test.
- 2 Nie raportuj SBTM jako „klikania ad hoc”: charter, time-box, notes, debrief.

8. Pairwise, orthogonal arrays, base-choice

Definicja:

Pairwise gwarantuje, że każda PARA parametrów wystąpi w co najmniej jednym TC — zwykle łapie większość defektów interakcji przy logarytmicznej liczbie przypadków. Orthogonal arrays dodają kontrolowaną siłę t (interakcje 2-, 3-, 4-krotne). Base-choice to odwrócona strategia: jeden „normalny” wektor bazowy, potem zmieniamy JEDEN parametr na raz — tanio i czytelnie dla review.

Stosuj, gdy...	NIE stosuj, gdy...
wiele opcji konfiguracyjnych (mode × currency × scenario w Checkout Lab)	parametry mają <3 wartości lub są niezależne — pełne pokrycie tanie
budżet TC ograniczony, a defekty historyczne = interakcje par	krytyczna jest interakcja trój-/czterokrotna OA siła t=3/4

```
// tests-pom/methods/pairwise/CheckoutModeOutcome.ts
export const checkoutModeOutcomes = [
  { id: 'SCN-CPL-01', mode: 'ONLINE', action: 'approve', fulfillment: 'CONFIRMED' },
  { id: 'SCN-CPL-02', mode: 'ONLINE', action: 'lie', fulfillmentNot: 'CONFIRMED' },
  { id: 'SCN-CPL-03', mode: 'CASH', action: 'none', fulfillment: 'CONFIRMED' },
  { id: 'SCN-CPL-04', mode: 'ONLINE', action: 'decline', fulfillment: 'CANCELLED' },
  { id: 'SCN-CPL-06', mode: 'EXPIRED_LINK', action: 'open', testId: 'psp-link-expired' },
] as const // oracle = fulfillment-status, nigdy sam query status
```

9. Error guessing i syntax testing (dirty dozen)

Definicja:

Error guessing to systematyzacja doświadczenia: listę klasycznych defektów (lost update, BOLA, puste stringi, znaki specjalne) zamieniamy w checklistę ataku. Syntax testing podaje parserowi złośliwe dane: malformed ETag „stale-etag” musi dać 400, nie 412; „v99” to legalna składnia, ale stara wersja — 412. W labie klasy defektów mapują się na konkretne odpowiedzi API, więc każda heurystyka kończy się konkretnym TC.

```
// tests-pom/methods/ep-bva/IfMatchAndKeyBoundaries.ts – dirty dozen na nagłówkach
export const authorizeHeaderBoundaries = [
  { id: 'SCN-LIF-01-ON', ifMatch: 'fresh', expectStatus: 200 }, // poprawna składnia + aktualna wersja
  { id: 'SCN-LIF-04', ifMatch: 'absent', expectStatus: 428 }, // brak naglowka
  { id: 'SCN-LIF-03', ifMatch: 'v99', expectStatus: 412 }, // legalna składnia, stara wersja
  { id: 'SCN-LIF-05', ifMatch: 'malformed', expectStatus: 400 }, // zła składnia: stale-etag
]

// REST Assured – BOLA maskowany 404 (autentyczny test tenant isolation)
MerchantApiTestSupport.requestWithToken(port, readerTokenB) // reader z merchant B
  .when()
  .get("/api/merchants/{merchantId}/payment-orders/{paymentOrderId}",
    merchantIdA, paymentOrderId) // zasob merchantu A
  .then()
  .statusCode(404) // NIE 403 – maskowanie
  .contentType(ContentType.JSON)
  .header("X-Correlation-ID", equalTo("corr-business-tenant-read"))
  .body("error", equalTo("not_found"));
```

10. Property-Based Testing (fast-check)

Definicja:

Zamiast pojedynczych przykładów definiujemy własność invariantu (np. „klucz idempotencji zawsze niepusty i 255 znaków”) i generator losowych danych (arbitrary); framework wykonuje 100 iteracji szukając kontrprzykładu. Po znalezieniu shrinkuje dane do minimalnego przypadku. W labie fast-check testuje czystą logikę frontendową wyekstrahowaną z komponentów, a jqwik jest planowany dla koperty Kafka v1 (E5).

```
// CreatePaymentOrderForm.property.test.ts (fragment)
it('Property 12a – initial Idempotency-Key is non-empty and ≤255 characters (≥100 iterations)', async () => {
  await fc.assert(
    fc.asyncProperty(formStateArb, async (state) => {
      const machine = new IdempotencyKeyStateMachine(state)
      expect(machine.currentKey.length).toBeGreaterThan(0)
      expect(machine.currentKey.length).toBeLessThanOrEqual(255)
    }),
    { numRuns: 100 },
  )
})
```

Stosuj, gdy...	NIE stosuj, gdy...
czysta funkcja/invariant bez I/O (mapery, walidatory, state machines)	test wymaga pełnego stacku HTTP+DB — tam RA/Testcontainers
chcesz wykryć rzadkie kombinacje danych (unicode, NaN, długie stringi)	własność jest trywialna ($x == x$) albo generator pomija klasy błędów

Ściąga: którą metodę na który problem

Problem	Metoda	Warstwa	Plik w repo
walidacja kwoty/zakresu	EP + BVA	REST Assured	methods/ep-bva/CaptureAmountPartitions.ts
nagłówki × akcja × stan	Tabela decyzyjna	RA + PW-API	methods/decision-table/IfMatchActionMatrix.ts
cykl życia encji	State Transition	RA (atomy) + E2E (P0)	methods/state/PaymentStatusMachine.ts
przepływ aktora end-to-end	Use Case / scenariusz	E2E molekuły	specs/merchants-concurrency.spec.ts
idempotencja, filtry, ETag	Metamorphic	REST Assured	methods/metamorphic/*.ts
multi-dimension cuts	Domain / CTM	dane deklaratywne	playbook 07 §ISTQB Advanced
opcje konfiguracji CPL	Pairwise / base-choice	RA + UI	methods/pairwise/CheckoutModeOutcome.ts
złośliwe dane wejściowe	Syntax / error guessing	REST Assured	methods/ep-bva/IfMatchAndKeyBoundaries.ts
inwarianty czystego kodu	Property-based	Vitest + fast-check	CreatePaymentOrderForm.property.test.ts
eksploracja pod niepewność	RST: SFDPOT/HICCUPPS/ SBTM	manual + charter	playbook 07 §RST

Jak o tym opowiedzieć na rozmowie (esencja playbooka 07):

„Foundation daje EP/BVA/DT. Na tym produkcie dokładam domain testing (amount × PIN × policy), classification tree, żeby nie eksplodować ról, cause-effect na 412/428/409, a strategię biorę z RST HTSM: najpierw SFDPOT i oracles HICCUPPS, potem dopiero tabela. Metamorphic zostawiam na relacje list (węższy filtr szerszy), nie na każdy przypadek.”

Zasady domowe labu obowiązujące przy wszystkich metodach:

- 1 Warstwa: REST Assured = atomy kontraktu; Playwright BFF-API = kontrakt przez proxy; E2E = tylko molekuły P0.
- 2 Dane: fabryki z unikalnymi referencjami per test; nigdy sleep-y — Awaitility/expect.poll.
- 3 Oracle: status + problem+json + stan DB; MR tam, gdzie oracle drogi.
- 4 Seed learningu (DataLearningDataset) nigdy nie przechodzi przez domain services — ADR 0001.
- 5 Wyłączenia suite'ów restkit/ i paymentsupport/ przy walidacjach backendowych.